Cape Peninsula
University of Technology

**University:** Cape Peninsula University of Technology

**Faculty:** Faculty of Engineering and the Built Environment

**Department:** Department of Electrical, Electronic

and Computer Engineering

**Project Title:** Student Records System

**Group Details:**


- Tshiamo Katlego Buhle Motloung – 260143103
- Msizi Ezra Mthethwa – 260615269
- Raisibe Letty Rakgotho – 261462954
- Mohau Tyron Ramashala – 260105341
- Abdurrahman Aziz – 221461833
- Mosweu Mathew Ramusi – 260854107
- Phuti Onthatie Shilajoe – 261454161
- Zayaan Galant – 230332307


**Lecturer:** Mr. Stephen Ekwe

**Date: 7 November 2026**


**CPUT copyright information**

The dissertation/thesis may not be published either in part (in scholarly, scientific or technical journals), or as a whole (as a monograph), unless permission has been obtained from the University

# Declaration

We, the members of group 1, declare that the contents of this report represent my own unaided work, and that the report has not previously been submitted for academic examination towards any qualification. Furthermore, it represents my own opinions and not necessarily those of the Cape Peninsula University of Technology.



Signed: Date: 7 November 2026

# Contents

Declaration ..... 2
Introduction ..... 4
Functional Requirements ..... 4
Sub-Questions ..... 4
Software Design Approach ..... 6
Planning stages: ..... 6
Tools and skills used ..... 6
The UML Diagram ..... 7
Programme Implementation ..... 8
The Graphical User Interface (GUI) ..... 8
Core Entity Classes and Data Models ..... 9
Testing ..... 13
Login Dashboard ..... 13
Lecturer Dashboard ..... 13
Admin Dashboard ..... 14
Authorities Dashboard ..... 15
Analysis of Results ..... 16
Conclusion ..... 17
References ..... 17

# Introduction

Lecturers, admins and authorities of a university institution frequently need to access and search through student records. The process of manually searching through these records is often tedious and time-consuming. A solution to this problem would be to create an automated system to replace this process. The automated system needs to store both personal and academic records for students within a database.

This project aims to design a Java program that enables users to insert, update, search and delete student records. The system needs to store personal and academic records for students within a database.

Different types of users will receive varying levels of access to the developed application namely:

- **Authorities:** Allowed access to all student records.
- **Administrators and Lecturers:** Restricted access, limited to registration and course enrolment.

# Functional Requirements

The primary objective of the automated software solution that allows users to:

- Enable the creation of new student records.
- Be able to store both the personal and academic records of each student.
- It must implement role-based access control: authorities receiving full access and admins given restricted access for registration and course enrolment.

## Sub-Questions

**How can we ensure authorities have full access while lecturers and admins are restricted to their specific functions?**

- By making use of the role-based access control, enforced through authentication controls and condition control states. Upon login, a user's access level is evaluated. Authorities are granted full access permissions while admins and lecturers have their UI components restricted.

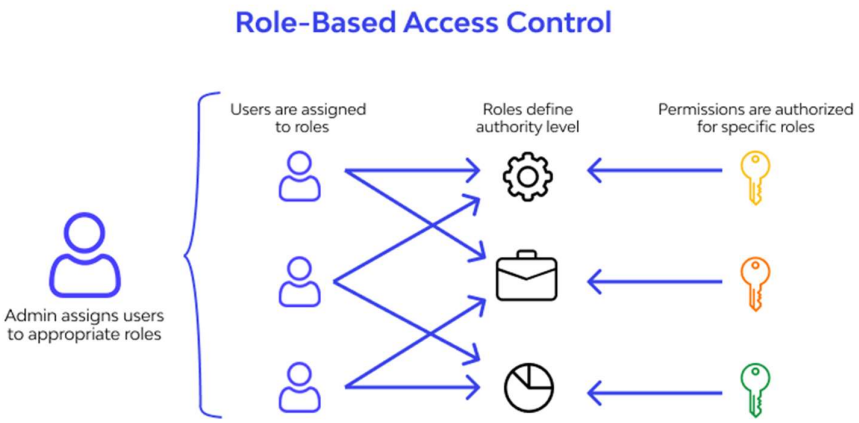



*Figure 1: Role based access control diagram (Lee, 2025)*

*Table 1: Role-based access table*

| User Role | Insert Record (create) | Update Record (Modify/Enroll) | Search Record (Read) | Delete Record (Delete) | Course Enrolment |
|---|---|---|---|---|---|
| Authority | Allowed | Allowed | Allowed | Allowed | Allowed |
| Admin | Denied | Restricted to enrolment only | Allowed | Denied | Allowed |
| Lecturer | Denied | Restricted to enrolment only | Allowed | Denied | Allowed |

**How can the application process the searching, insertion, and deletion of records?**

- The application utilizes a centralized *Datastore* architecture built with internal Java *ArrayLists* and object encapsulation. Using custom operation methods (i.e addStudent(), updateGrade() and student search filters) we eliminate the need for manually searching databases and manual recording.

**What tools can be used to create a secure, user-friendly interface?**

- With the use of Java Swing GUI components, the system is made up of text fields, tables and drop down menus (Oracle, 2019). Core object-orientated concepts - (i.e constructors and modifiers) to initial individual dashboards, record object fields and data encapsulation between entity classes (Deitel & Deitel, 2018) – work together to perform input validation and error handling to ensure data is secure.

# Software Design Approach

The student record management system was developed using the Waterfall Software Development Life Cycle (SDLC) (GeeksforGeeks, 2025). The model allowed the team to complete the project step-by-step from planning to implementation while allowing for constant testing throughout.

## Planning stages:

1. **Requirements analysis**: the project brief was studied to identify the required functions of the application as discussed above in the Introduction and Sub-questions.
2. **System Design**: the UML class diagram was designed to identify the main classes and relationships within the system before pseudocode was drafted and implemented.
3. **Implementation**: The system was developed using Java and object-orientated programming principles. The Java JFrame Forms were used to create the graphical user interface (Oracle, 2019).
4. **Testing:** the programme was tested. Functions such as login, student administration, student searches and grade updates were tested to ensure that they worked correctly. Input validation and error handling was also test.
5. **Documentation:** members made use of GitHub repository to commit code and upload research documentation throughout the process along for all members to remain organised throughout the project.

GitHUB Repository link - Student Records System

## Tools and skills used

**UML modelling tools** were used to design the UML class diagram during the planning stage.

**Apache NetBeans IDE** was used by individual team member to write, compile, debug and test the Java source code. **Java JFrame Forms** were used to create the graphical user interface, allowing users to interact easily with the system.

**GitHub** was used for version control and contribution management. Team members uploaded their individual code files to the repository and constantly tested the program and debugged the completed program. The repository was also used to monitored project progress and maintained different versions of the project as time passed.

These tools helped improve collaboration, organisation and the overall quality of the project.

# The UML Diagram

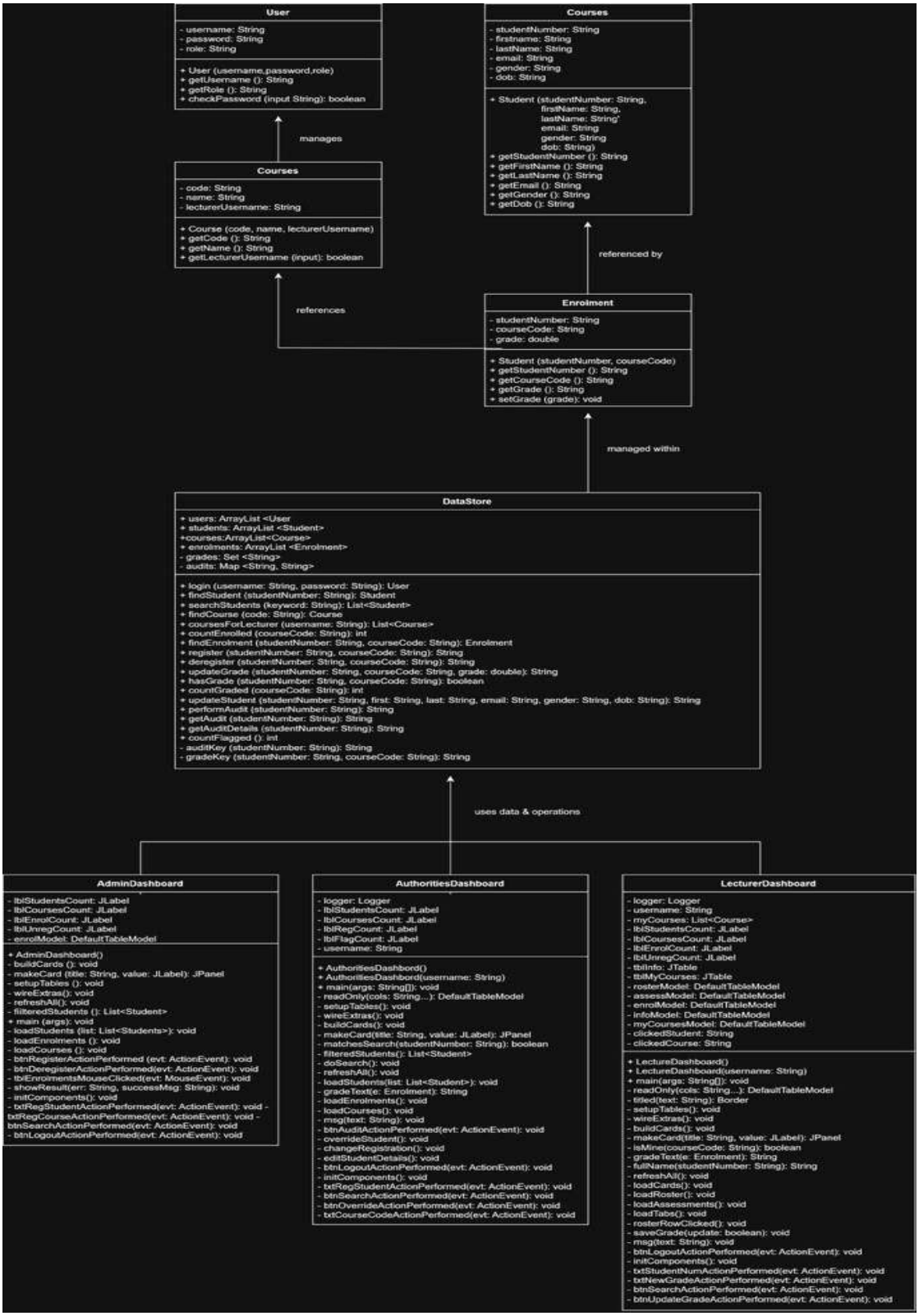


*Figure 2: The UML class diagram*

The UML Class Diagram was developed during the design phase to provide a visual representation of the structure of the Student Record Management System. The diagram shows the relationships between users, students, courses, subjects, enrolments and the central *DataStore* class.

It also served as a blueprint during programme implementation improving the overall efficiency of the coding process.

# Programme Implementation

## The Graphical User Interface (GUI)

The GUI was developed using Java Swing and JFrames forms. This approach allowed for an interactive and user-friendly system which allows users to perform tasks through buttons, text fields, tables and menu options (Oracle, 2019).

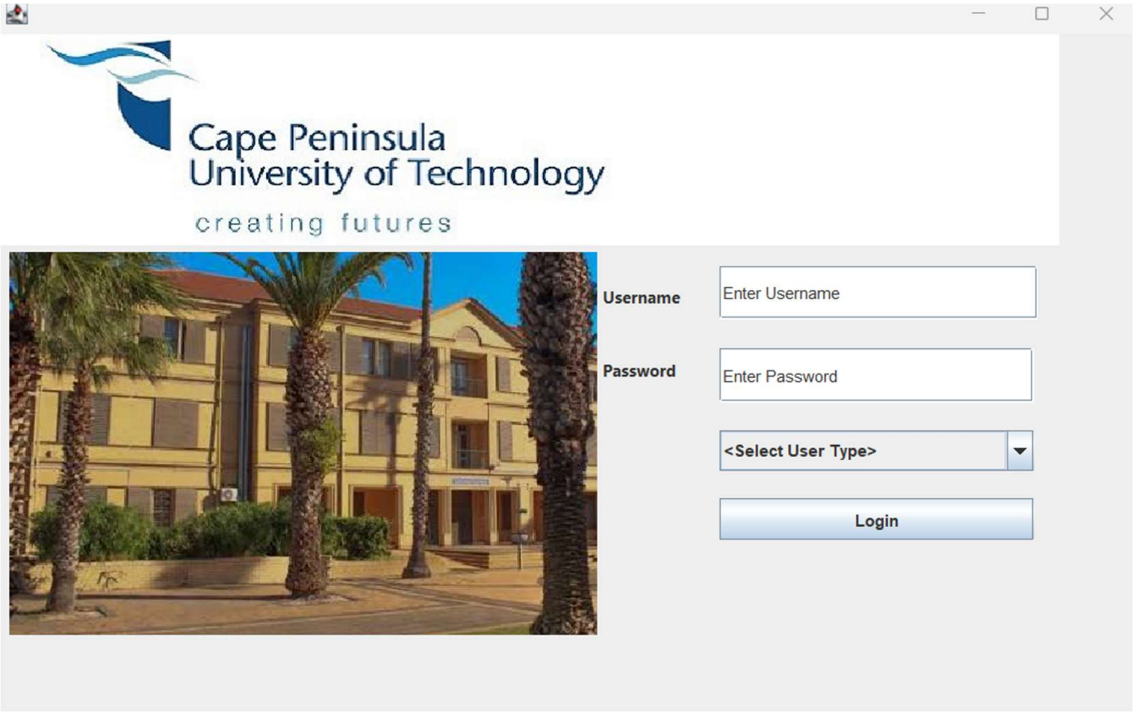



*Figure 3: The login Dashboard*

*Table 2: User credential table*

| User type | Username | Password |
|---|---|---|
| Lecturer | lecturer1 | lect123 |
| | lecturer2 | lect456 |
| Admin | admin1 | admin123 |
| Authority | authority1 | auth123 |

## Core Entity Classes and Data Models

***DataStore.java:*** Acts as the centralised data management component of the system. These contain pre-populated sample accounts and student records while housing operational logic.

The database was hardcoded into the *DataStore* class. Student records, user accounts, courses, and enrolment information are stored and managed through collections. This structure enables efficient searching, updating, and deletion of records while maintaining data organization throughout the application.

**User.java:** manages staff authentication credentials and system roles (authority, admin and lecturer). It provides methods (*checkPassword()* and *getRole()*) to govern system permissions.

```
package model;

public class User {
    private String username, password, role; // role: LECTURER, ADMIN, AUTHORITY

    public User(String username, String password, String role) {
        this.username = username;
        this.password = password;
        this.role = role;
    }

    public String getUsername() { return username; }
    public String getRole() { return role; }
    public boolean checkPassword(String input) { return password.equals(input); }
}
```

*Figure 4: User.java code*

**Student.java:** encapsulates demographic and personal details, storing attributes such as students first names, last names, student numbers, etc.

```java
package model;

public class Student {
    private String studentNumber, firstName, lastName, email, gender, dob;

    public Student(String studentNumber, String firstName, String lastName,
                   String email, String gender, String dob) {
        this.studentNumber = studentNumber;
        this.firstName = firstName;
        this.lastName = lastName;
        this.email = email;
        this.gender = gender;
        this.dob = dob;
    }

    public String getStudentNumber() { return studentNumber; }
    public String getFirstName() { return firstName; }
    public String getLastName() { return lastName; }
    public String getEmail() { return email; }
    public String getGender() { return gender; }
    public String getDob() { return dob; }
}
```

*Figure 5: Student.java code*

**Course.java & Subject.java**: Model the academic curriculum, tracking programme codes, module titles, and assigned lecturer usernames.

```java
/**
 * Represents an academic programme/course, for example
 * BTech Computer Engineering. Individual subjects belong in Module.
 */
public class Course {

    private final String code;
    private final String name;

    public Course(String code, String name) {
        this.code = code;
        this.name = name;
    }

    public String getCode() {
        return code;
    }

    public String getName() {
        return name;
    }

    /**
     * Compatibility method for older dashboard code.
     * Lecturer assignment now belongs to each Module, not the programme.
     */
    @Deprecated
    public String ~~getLecturerUsername~~() {
        return "See module assignments";
    }
```

*Figure 6: Course.java code*

```java
package model;

/** Represents a subject/module that belongs to an academic Course. */
public class Subject {

    private final String code;
    private final String name;
    private final String courseCode;
    private final String lecturerUsername;

    public Subject(String code, String name, String courseCode,
                   String lecturerUsername) {
        this.code = code;
        this.name = name;
        this.courseCode = courseCode;
        this.lecturerUsername = lecturerUsername;
    }

    public String getCode() {
        return code;
    }

    public String getName() {
        return name;
    }

    /** Returns the code of the parent academic programme/course. */
    public String getCourseCode() {
        return courseCode;
    }

    public String getLecturerUsername() {
        return lecturerUsername;
```

*Figure 7: Subject.java code*

**Enrolment.java**: Links students to specific modules, storing the studentNumber, moduleCode, and the associated grade.

```java
package model;

/** Represents a subject/module that belongs to an academic Course. */
public class Subject {

    private final String code;
    private final String name;
    private final String courseCode;
    private final String lecturerUsername;

    public Subject(String code, String name, String courseCode,
                   String lecturerUsername) {
        this.code = code;
        this.name = name;
        this.courseCode = courseCode;
        this.lecturerUsername = lecturerUsername;
    }

    public String getCode() {
        return code;
    }

    public String getName() {
        return name;
    }

    /** Returns the code of the parent academic programme/course. */
    public String getCourseCode() {
        return courseCode;
    }

    public String getLecturerUsername() {
        return lecturerUsername;
    }
}
```

*Figure 8: Enrolment.java code*

Several important Java programming concepts were incorporated during implementation. Constructors were used to initialize objects when new records were created, while access modifiers such as private and public were applied to ensure data encapsulation. Control statements including if-else statements, loops, and switch statements were used to control program execution and validate user input. Exception handling mechanisms were implemented to improve system reliability and prevent application failures caused by invalid operations.

The completed implementation provides functionality for student management, course management, enrolment processing, grade management, searching, registration, deregistration, and user authentication through an interactive graphical environment.

# Testing

Testing was conducted throughout the development process to ensure that all functions operated according to the requirements. The testing for the completed programme was heavily focused on the role-based access table (*Table 2*).

## Login Dashboard

The login functionality was tested to verify whether legitimate users could access the system and whether unauthorized users were denied access.

Even if the user inputs valid credentials, if the username does not match the role type, users will not be able to access any information, and an error message will be displayed.

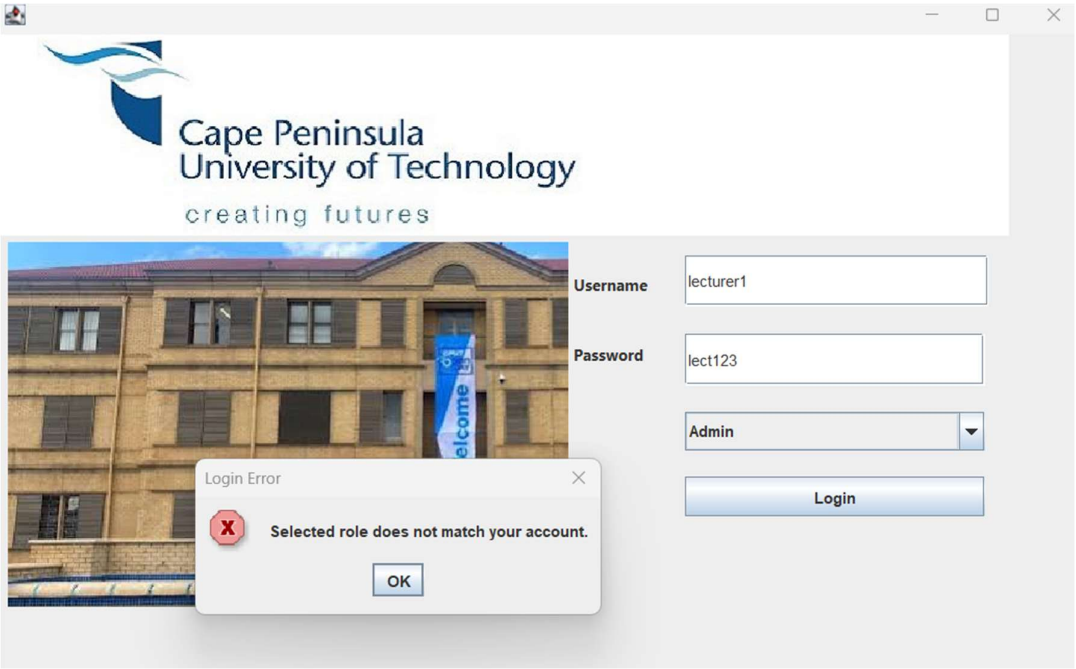



*Figure 9: Login Dashboard with mismatched roles and credentials*

## Lecturer Dashboard

Lecturers can view basic information for their students and update grades in the grade control panel shown below. Functional testing validated that lecturers can accurately record and update student grades within their assigned modules. Concurrently, security verification confirmed that system restrictions successfully prevent lecturers from executing unauthorized system actions, such as deleting student database files or overriding high-level institutional credentials, ensuring strict adherence to the project's Role-Based Access Control (RBAC) policy.

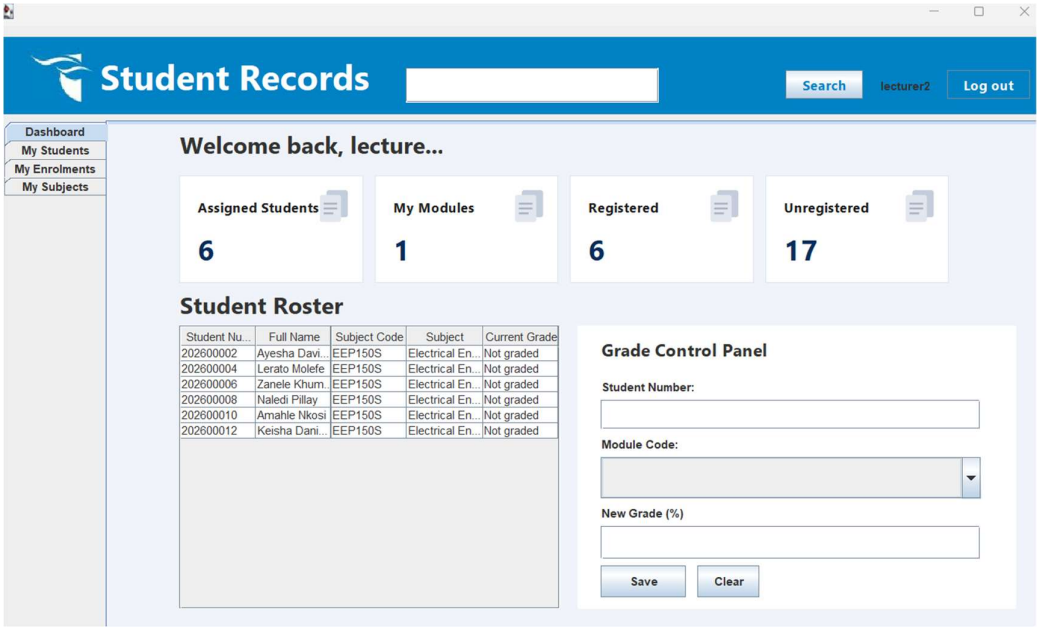



*Figure 10:Lecturer dashboard*

## Admin Dashboard

The admin dashboard allows users to search and view student information, course enrolments and register and deregister students for modules.

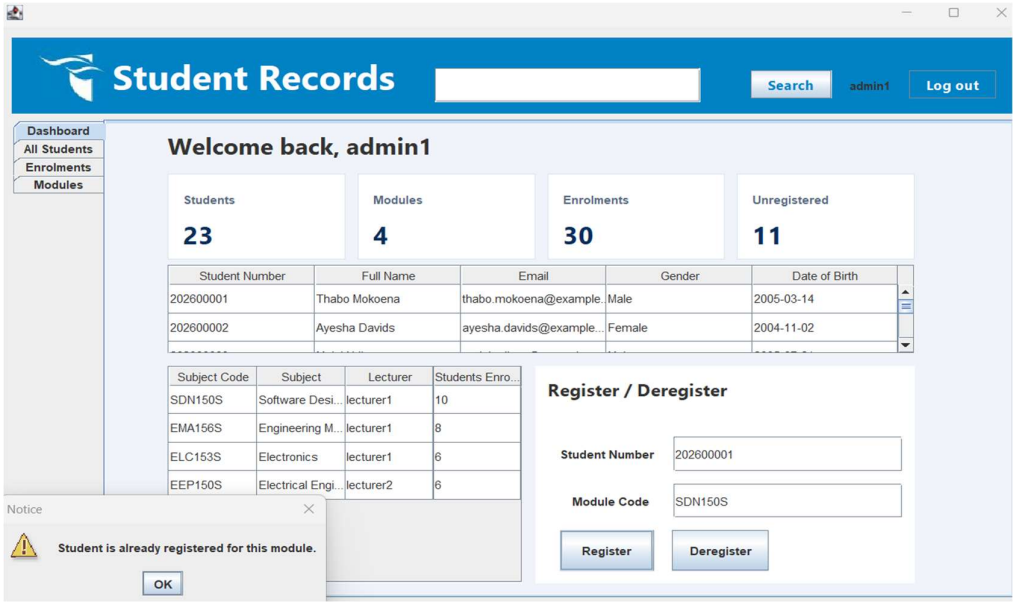



*Figure 11: Admin dashboard - error message for student already enrolled.*

Admins are also able to view modules and the lecturers responsible for them. Updates to grades made by lecturers are also visible within the Enrolments tab. permission validation tests verified that

while administrators retain full capability to manage student registrations, the system's control-flow restrictions successfully block unauthorized administrative attempts to delete student profile records or modify restricted personal data fields.

## Authorities Dashboard

As mentioned previously, authorities have all the access permissions.

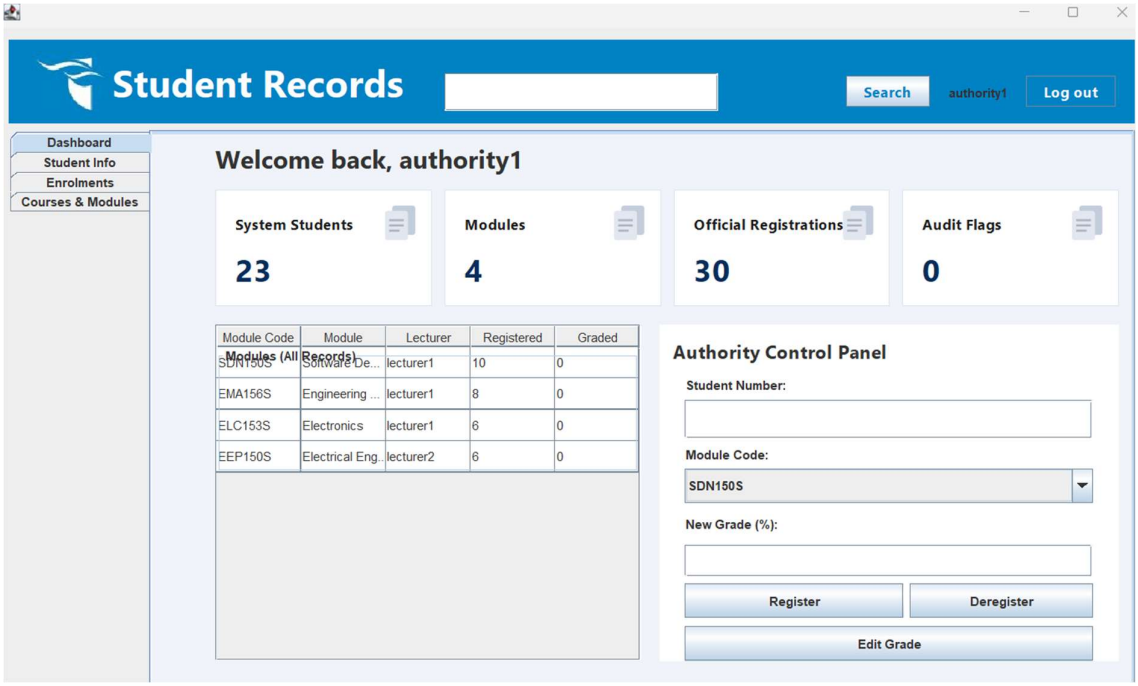



*Figure 12: Authorities Dashboard*

Functional testing of the Authority control panel confirmed that authorized personnel can seamlessly execute full administrative operations. Allowing authorities to register students for specific modules, deregister enrolments.

*Table 3: System testing summary table*

| Test Case | Expected result | GUI outcome | Status |
|---|---|---|---|
| User Login | User access correct dashboard | Dashboard displayed correctly | Pass |
| | User access incorrect dashboard | Error message outputed | Pass |
| Search Student | Student record displayed | Student record displayed | Pass |

| | | | |
|---|---|---|---|
| Register Student | Student enrolled successfully | Enrolment successful | Pass |
| Deregister Student | Student removed from course | Deregistration successful | Pass |
| Update Student Results | Student details updated | Information updated correctly | Pass |

The testing process confirmed that all major features functioned correctly and that validation mechanisms prevented invalid data from affecting system performance.

# Analysis of Results

The results obtained from testing demonstrate that the Student Record Management System successfully fulfilled the primary objectives of the project. The application was able to authenticate users correctly and provide access according to predefined user roles. The role-based structure improved security and ensured that users could only access functions relevant to their responsibilities.

The student management functions operated effectively, allowing users to create, search, update, and manage student records efficiently. Search operations returned accurate results and significantly reduced the time required to locate student information when compared to manual record management methods.

Course registration and deregistration functions performed successfully and updated enrolment information accurately. The system also enabled lecturers and authorities to monitor and manage academic information through dedicated dashboard interfaces.

The graphical user interface improved usability by providing clear navigation structures, organized data presentation, and interactive controls. Dashboard statistics provided users with an overview of student, course, and enrolment information, improving administrative monitoring and decision-making.

Overall, the implemented system demonstrated reliability, responsiveness, and functionality throughout the testing process.

# Conclusion

The Student Record Management System successfully achieved its objective of providing an efficient and user-friendly platform for managing student records within the CPUT Faculty of Engineering and the Built Environment.

The project demonstrated the practical application of object-oriented programming principles using classes, constructors, access modifiers, control statements, and exception handling techniques. The implementation of Java Swing and JFrame forms enabled the development of an interactive graphical user interface that supports efficient management of students, courses, enrolments, and academic information.

Although the current implementation provides an effective solution, future enhancements such as database integration, cloud-based deployment, and advanced security features could further improve the system's performance and scalability. Overall, the project demonstrates how software solutions can improve efficiency, accuracy, and accessibility in academic record management.

# References


Deitel, P.J. and Deitel, H.M. (2018). *Java : How to program : Early objects*. 11th Edition edn. New York, Ny Pearson.

GeeksforGeeks (2025). *Software development life cycle (SDLC)*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/software-engineering/software-development-life-cycle-sdlc/ [Accessed 15 Oct. 2026].

Janssen, T. (2023). *OOP concept for beginners: What is encapsulation*. [online] Stackify. Available at: https://stackify.com/oop-concept-for-beginners-what-is-encapsulation/ [Accessed 1 Nov. 2026].

*Java HashSet tutorial*. (2015). [Video] Directed by Coders Campus. YouTube. Available at: https://www.youtube.com/watch?v=WPcKwA5WF7s [Accessed 27 Oct. 2026].

*Java swing UI design - login and register*. (2021). [Video] Directed by Ra Ven. YouTube. Available at: https://youtu.be/ZUGkQHtLS4U [Accessed 1 Nov. 2026].

Lee, I. (2025). *What is RBAC role-based access control ❓ types and benefits.* [online] www.wallarm.com. Available at: https://www.wallarm.com/what/what-exactly-is-role-based-access-control-rbac [Accessed 29 Oct. 2026].

Oracle (2019). *Trail: Creating a GUI with JFC/swing (the java™ tutorials)*. [online] Oracle.com. Available at: https://docs.oracle.com/javase/tutorial/uiswing/ [Accessed 30 Oct. 2026].

*Uso de JOptionPane.showConfirmDialog en java*. (2020). [Video] Directed by Fredy Geek. YouTube. Available at: https://www.youtube.com/watch?v=GwjUKVeVtCo [Accessed 31 Oct. 2026].